# 算法笔记：时间复杂度与时限判定

## 01 评测机制与 TLE 说明

在 OJ（在线评测系统）中，代码通过样例却在提交后出现 **TLE（Time Limit Exceeded）**，通常与以下评测机制有关：

### 1. 时间限制的定义

题目侧边栏给出的"时间限制"（如 1000 ms），是指程序在运行**单个测试点**时所允许占用的最大 CPU 时间。若程序在某一组测试数据上的执行耗时超过该阈值，系统将终止程序并判定该测试点 TLE。

### 2. 样例与测试点的差异

题目给出的输入样例通常规模较小，用于演示题意；而后台评测点包含针对数据规模上限（如 $N = 10^5$ 或更大）设计的边界用例。算法在小数据下耗时可忽略，但在满规模输入下运算量激增，就会超出时限。

## 02 计算机运算能力与 $10^8$ 经验阈值

### 1. 基本运算（一次操作）

在渐进分析中，"一次基本运算"指能在固定且极短的时钟周期内完成的原子操作，其耗时视为常数阶 $O(1)$：

- **基本算术与位运算**：`+`、`-`、`*`、`/`、`%`、`>>`、`<<`、`^` 等。
- **关系与逻辑运算**：`==`、`!=`、`<`、`>`、`&&`、`||` 等。
- **简单变量赋值与寻址**：如 `int t = i`、访问数组单点元素 `a[i]`。

> **注意**：标准库函数（如 `std::sort` 耗费约 $O(N \log N)$）、容器扩容、长字符串逐字符比较等均非单次操作，内部包含大量基本运算。

### 2. $10^8$ 运算量基准

现代评测环境在 1 秒时限内，通常能够执行约 $10^7 \sim 10^8$ **次基本运算**。受指令类型、访存连续性与输入输出常数影响，实际可承载量有所浮动，但在算法设计时通常采用 $10^8$ 作为保守安全阈值：

- **总运算步数** $\leq 10^8$：通常能在 1 秒内执行完毕。
- **总运算步数** $\geq 10^9$：通常会超出 1 秒，导致 TLE。

### 3. 代码运算量估算

对于嵌套循环结构，可使用以下公式粗略估算代码的总运算步数：

> 总运算量 ≈ 外层循环次数 × 内层循环次数 × 单次循环体内部的操作数

# 03 时间复杂度（大 $O$ 记号）与分析原则

## 1. 基本概念

时间复杂度用于描述**基本运算次数随输入数据规模 $N$ 增长的渐进趋势**，而非具体的毫秒数。

- **只保留最高阶项**：当 $N \to \infty$ 时，最高阶项决定增长速率，低阶项的影响微弱。
- **忽略常数系数**：常数倍数不改变函数随规模增长的阶数本质（例如 $2N^2$ 与 $N^2$ 均属于二次方增长）。

## 2. 复杂度分析的四大原则

### 原则一 常数与固定循环归为 $O(1)$（常数原则）

- **逻辑**：代码执行步数为一个固定的常数，不随输入规模 $N$ 的改变而改变。
- **示例**：
  - 直接代数公式计算：`ans = (long long)N * (N + 1) / 2;`，固定为 3 次基本操作。
  - 循环次数写死：如 `for (int i = 1; i <= 7; i++)`，执行次数固定，记为 $O(1)$。

### 原则二 嵌套结构做乘法（乘法原则）

- **单层循环**：`for (int i = 0; i < N; i++)` 执行 $N$ 次，复杂度为 $O(N)$。
- **双层完整循环**：外层 $N$ 次，内层 $N$ 次，总步数约为 $N \times N = N^2$，复杂度为 $O(N^2)$。
- **等差递减嵌套**： 执行总次数为 $\Sigma_{i=1}^{N}(N-i+1) = \frac{N(N+1)}{2} = \frac{1}{2}N^2 + \frac{1}{2}N$。略去系数与低阶项后，复杂度仍为 $O(N^2)$。（估算技巧：内层从 $N$ 递减到 1，均值约为 $\frac{N}{2}$，乘积 $N \times \frac{N}{2} = \frac{N^2}{2}$ 即为 $O(N^2)$）。

```
for (int i = 1; i <= N; i++) {
    for (int j = i; j <= N; j++) { ... }
}
```

- **多重嵌套**：三重循环严格枚举三元组，步数约为 $\frac{N^3}{6}$，复杂度为 $O(N^3)$。

### 原则三 步长按比例缩减看对数（对数原则 $O(\log N)$）

- **逻辑**：循环变量每次并非线性自增（+1），而是乘除一个常数因子（如 `/= 2` 或 `/= 10`）。
- **示例**：
  - 折半循环：`while (N > 0) { N /= 2; }`，循环次数为 $\lfloor \log_2 N \rfloor$，属于 $O(\log N)$。
  - 数位剥离：`while (t > 0) { t /= 10; }`，执行次数等于该数的十进制位数 $\lfloor \log_{10} N \rfloor + 1$。
  - 嵌套对数：外层单循环 $N$ 次，内层进行数位拆解或二分检索，总时间复杂度为 $O(N \log N)$。

### 原则四 并列结构取最高阶（加法原则）

- **逻辑**：多个代码段依次独立执行时，总体渐进复杂度取决于阶数最高的一段。
- **示例**：先执行一个 $O(N)$ 循环，紧接一个 $O(N^2)$ 循环，总步数为 $N + N^2$，整体复杂度为 $O(N^2)$。

## 3. 代码复杂度判定的三步流程

01 **圈定规模变量**：确认代码中约束循环边界的核心输入变量（如 $N, M$）。

02 **定位深层结构**：找出执行频次最高的内层核心循环或递归主体。

03 **计算并化简**：将嵌套层级的循环步数相乘，去除常数系数与并列低阶项，保留最高阶项作为最终结果。

# 04 由数据规模 $N$ 预估算法复杂度

在编写代码前，观察题目给出的最大数据规模 $N$，结合 $10^8$ 算力上限，可以直接推导可行的算法复杂度范围，排除会导致超时的设计方案。

## 数据规模 $N$ 与允许的最大时间复杂度对照表

(以标准评测时限 1000 ms、约 $10^8$ 次基本运算为参考基准)

| 题目给定的规模 N | 允许的最高时间复杂度 | 代入极值后的运算量 | 适用算法与数据结构 | 1 秒内耗时估算 |
|---|---|---|---|---|
| $N \le 10$ | $O(N!)$ | $10! \approx 3.6 \times 10^6$ | 全排列枚举、递归回溯暴搜 | $\approx 0.05$s |
| $N \le 20$ | $O(2^N)$ | $2^{20} \approx 10^6$ | 子集枚举、状态压缩 DP | $\approx 0.01$s |
| $N \le 100 \sim 300$ | $O(N^3)$ | $100^3 = 10^6$ | 三重嵌套循环、Floyd 最短路 | $\approx 0.01 \sim 0.05$s |
| $N \le 1{,}000 \sim 2{,}000$ | $O(N^2)$ | $1000^2 = 10^6$ | 双重嵌套循环、二维动态规划 | $\approx 0.01$s |

| 题目给定的规模 N | 允许的最高时间复杂度 | 代入极值后的运算量 | 适用算法与数据结构 | 1 秒内耗时估算 |
| --- | --- | --- | --- | --- |
| $N \le 10^5 \sim 10^6$ | $O(N \log N)$ | 约 $1.7 \times 10^6 \sim 2 \times 10^7$ | 快速排序、归并排序、二分查找、分治 | $\approx 0.05 \sim 0.2$s |
| $N \le 10^7 \sim 10^8$ | $O(N)$ | 约 $10^7 \sim 10^8$ | 单层循环线性扫描、双指针、前缀和、计数桶 | $\approx 0.05 \sim 0.1$s |
| $N \ge 10^9$ | $O(\sqrt{N})$ 或 $O(\log N)$ / $O(1)$ | 根号约 $3 \times 10^4$；对数约 30 | 试除法判素数（$\sqrt{N}$）、辗转相除（gcd）、快速幂、数学公式 | $< 1$ms |
| 与 $N$ 无关 | $O(1)$ | 固定常数步 | 条件分支、代数运算、固定次数循环 | $< 0.001$ms |

## 实操判定准则

- $N \le 1000$：双重循环（$O(N^2)$）运算量在 $10^6$ 级别，处于安全范围内。
- $N \ge 10^5$：双重循环理论运算量达 $10^{10}$，必然超出 $10^8$ 限制；需将复杂度降至 $O(N \log N)$ 或 $O(N)$。
- $N \ge 10^9$：单层线性循环 $O(N)$ 将超出时限，仅能采用 $O(\sqrt{N})$、对数级算法或 $O(1)$ 代数解法。